

Fundamental of Digital Logic Design

EEE240

Lab Manual



Name	
Registration Number	
Semester	
Instructor's Name	

Introduction

This is the Lab Manual for EEE240 Fundamentals of Digital Logic Design. The labs constitute almost 33 % of the total marks for this course.

During the labs you will work in groups (no more than three students per group). You are required to complete the 'Pre-Lab' section of the lab before coming to the lab. You will be graded for this and the 'In-Lab' tasks during the in-lab viva. You will complete the 'post-Lab' section of each lab before coming to next week's lab.

You are not allowed to wander in the lab or consult other groups when performing experiments. Similarly, the lab reports must contain original efforts. CUI has a zero tolerance anti-plagiarism policy.

Apart from these weekly labs you will complete two projects. Lab mid-term will be conducted and graded as 25% marks, respectively. Final Project / Final Exam which will be graded as Lab Terminal. The grading policy is already discussed in the Course Description File.

Acknowledgement

This is the initial draft of the EEE240 Fundamentals of Digital Logic Design manual. It is adapted from the EEE-241 Digital Logic Design Lab Manual, which was originally created by Dr. M. Faisal Siddiqui in 2014 and has since been updated by various faculty members in the ECE department.

The EEE241 course was designed for Electrical and Computer Engineering students with a strong background in circuit and hardware design, resulting in a lab curriculum focused heavily on hardware ICs and FPGA-based implementations. In contrast, students from other departments may lack these specific skills. As a result, the hardware implementation components in the EEE240 course have been minimized and are now focused primarily on simulation.

The first version of the EE240 manual was completed in the Spring 2024 session.

History of Revision

Version and Date of Issue	Team	Comments
Version 1. July 2024	Dr. Irfan Ullah Khattak	This is the first editable draft of EEE240 Fundamental of Digital Logic Design that was adopted from EEE241 Digital Logic Design lab manual. For comments and suggestions please contact: irfan_ullah@comsats.edu.pk

Safety Precautions

- Be calm and relaxed, while working in lab.
- First check your measuring equipment.
- When working with voltages over 40 V or current over 10 A, there must be at least two people in the lab at all time.
- Keep the work area neat and clean.
- Be sure about the locations of fire extinguishers and first aid kit.
- No loose wires or metals pieces should be lying on the table or near the circuit.
- Avoid using long wires, that may get in your way while making adjustments or changing leads.
- Be aware of bracelets, rings, and metal watch bands (if you are wearing any of them). Do not wear them near an energized circuit.
- When working with energized circuit use only one hand while keeping rest of your body away from conducting surfaces.
- Always check your circuit connections before power it ON.
- Always connect connection from load to power supply.
- Never use any faulty or damaged equipment and tools.
- If an individual comes in contact with a live electrical conductor.
 - Do not touch the equipment, the cord, the person.
 - Disconnect the power source from the circuit breaker and pull out the plug using insulated material.

Table of Contents

1. Introduction.....	2
Acknowledgement.....	3
History of Revision.....	3
2. Safety Precautions.....	4
3. Table of Contents	5
1. LAB #01: Introduction to Basic Logic Gate and Proteus Simulation.....	8
Objective	8
Pre-Lab:.....	8
Post-Lab Tasks:.....	19
Critical Analysis/Conclusion	20
2. LAB #02: Boolean Function Implementation using Universal Gates	21
Objectives	21
Pre-Lab:.....	21
In lab:	21
Post-Lab:	28
Critical Analysis/Conclusion	30
3. LAB #03: Design of Combinational Circuits and Logic Minimization Using Boolean Function	31
Objective	31
In-Lab:.....	31
Post-Lab:	32
Critical Analysis/Conclusion	33
4. LAB #04: Design and Implementation of Boolean Functions by Standard Forms	34
Objective	34
Pre-Lab:.....	34
Pre-Lab Tasks:	38
In-Lab Tasks:	39
Post-Lab Tasks:.....	41
Critical Analysis/Conclusion	43
5. LAB #05: Logic Minimization of Complex Functions using Automated Tools	44
Objective	44
Pre-Lab:.....	44

In-Lab:.....	45
Post-Lab Tasks:	49
Critical Analysis/Conclusion	49
6. LAB #06: Design 4-bit Binary to Gray Code Converter	51
Objective	51
Pre-Lab:.....	51
In-Lab:.....	54
Post-Lab Task:	54
Critical Analysis/Conclusion	55
7. LAB #07: Design and Implementation of $n - bit$ Adder/Subtractor.....	56
Objectives	56
Pre-Lab:.....	56
In-Lab Tasks:	57
Post-Lab:	60
Critical Analysis/Conclusion	62
8. LAB #08: Design and Implementation of $n - bit$ Binary Multiplier	63
Objective.....	63
Pre-Lab:.....	63
In-Lab Task 1: Implement $4 - bit$ by $2 - bit$ binary multiplier	65
In-Lab Task 2: Implement $4 - bit$ by $3 - bit$ binary multiplier	65
Post-Lab Tasks:.....	65
Critical Analysis/Conclusion	66
9. LAB #09: Design and Implementation of BCD to 7-Segment Decoder	67
Objective	67
Pre-Lab:.....	67
In-Lab Task : Design and implementation of a BCD to 7-Segment decoder on FPGA	68
Post-Lab Task:	69
Critical Analysis/Conclusion	71
10. Lab #10: Design and Implementation of a Sequence Detector using Mealy/Moore Machine.....	72
Objective	72
Pre-Lab:.....	72
In-Lab: Design and Implementation of a Sequence Detector using Mealy Machine	73

Post-Lab Tasks:.....	76
Critical Analysis/Conclusion	76

LAB #01: Introduction to Basic Logic Gate and Proteus Simulation

Objective

Part 1

To know about the basic logic gates, their truth tables, input-output characteristics and analyzing their functionality. Introduction to logic gates.

Part 2

Learn to use Proteus Software for Simulation of Digital Logic Circuits.

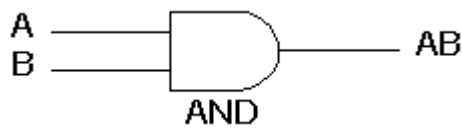
Pre-Lab:

Background Theory:

The Digital Logic Circuits can be represented in the form of **(1) Boolean Functions**, **(2) Truth Tables**, and **(3) Logic Diagram**. Digital Logic Circuits may be practically implemented by using electronic gates. The following points are important to understand.

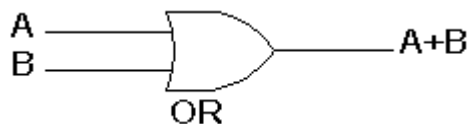
- Electronic gates are available in the form of Integrated Circuits (ICs) and they require a power.
- Supply Gate INPUTS are driven by voltages having two nominal values, e.g. 0V and 5, 12V representing logic 0 and logic 1 respectively.
- The OUTPUT of a gate provides two nominal values of voltage only, e.g. 0V and 5, 12V representing logic 0 and logic 1 respectively. In general, there is only one output to a logic gate except in some special cases.
- Truth tables are used to help show the function of a logic gate in terms of input values combination with the desired output.
- Logic Diagram is used to represent the Digital Logic Circuit in the form of symbols connected with each other.
- Digital Logic Circuits can be simulated in the virtual environment called simulation software

The basic operations are described below with the aid of Boolean function, logic symbol, and truth table.

AND gate:*Figure 1.1: AND gate**Table 1.1: Truth Table of 2 input AND gate*

<i>A</i>	<i>B</i>	<i>F = A . B</i>
0	0	0
0	1	0
1	0	0
1	1	1

The AND gate is an electronic circuit that gives a **high** output (1) only if **all** its inputs are high. A dot (.) is used to show the AND operation i.e. $A.B$. Bear in mind that this dot is sometimes omitted i.e. AB .

OR gate:*Figure 1.2: OR gate**Table 1.2: Truth Table of 2 input OR gate*

<i>A</i>	<i>B</i>	<i>F = A + B</i>
0	0	0
0	1	1
1	0	1
1	1	1

The OR gate is an electronic circuit that gives a high output (1) if **one or more** of its inputs are high. A plus (+) is used to show the OR operation.

NOT gate

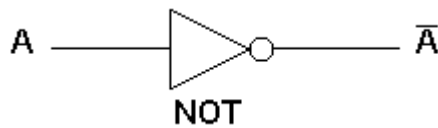


Figure 1.3: NOT gate

Table 1.3: Truth Table of NOT gate

A	$F = \bar{A}$
0	1
1	0

The NOT gate is an electronic circuit that produces an inverted version of the input at its output. It is also known as an *inverter*.

NAND gate



Figure 1.4: NAND gate

Table 1.4: Truth Table of 2 input NAND gate

A	B	$F = \overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

This is a NOT-AND gate which is equal to an AND gate followed by a NOT gate. The output of NAND gate is **high** if **any** of the inputs are low. The symbol is an AND gate with a small circle on the output. The small circle represents inversion.

NOR gate

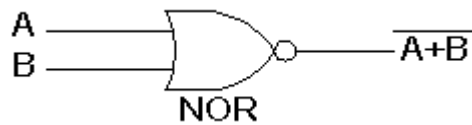


Figure 1.5: NOR gate

Table 1.5: Truth Table of 2 input NOR gate

A	B	$F = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

This is a NOT-OR gate which is equal to an OR gate followed by a NOT gate. The output of NOR gate is **low** if **any** of the inputs are high. The symbol is an OR gate with a small circle on the output. The small circle represents inversion.

XOR gate

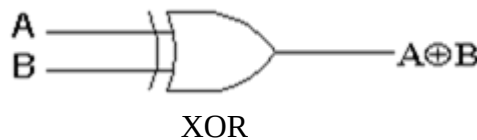


Figure 1.6: XOR gate

Table 1.6: Truth Table of 2 input XOR gate

A	B	$F = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

The '**Exclusive-OR**' gate is a circuit which will give a high output if **odd** number of inputs are high. An encircled plus sign " $\oplus$ " is used to show the EOR operation.

XNOR gate

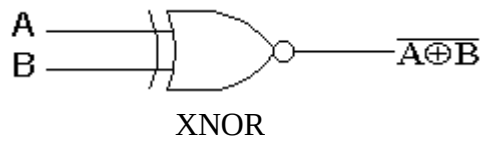


Figure 1.7: XNOR gate

Table 1.7: Truth Table of 2 input XNOR gate

A	B	$F = \overline{A \oplus B}$
0	0	1
0	1	0
1	0	0
1	1	1

The 'Exclusive-NOR' gate circuit does the opposite to the XOR gate. It will give a high output if **even** number of inputs are high. The symbol is an XOR gate with a small circle on the output. The small circle represents inversion.

In-lab Task 1:

Verify all gates using their ICs on KL-31001 Digital Logic Lab trainer

Table 1.8: Observation Table for different gates

INPUTS		OUTPUTS					
A	B	AND	OR	XOR	$NAND$	NOR	$XNOR$
0	0						
0	1						
1	0						
1	1						

Table 1.9: Observation Table for NOT gate

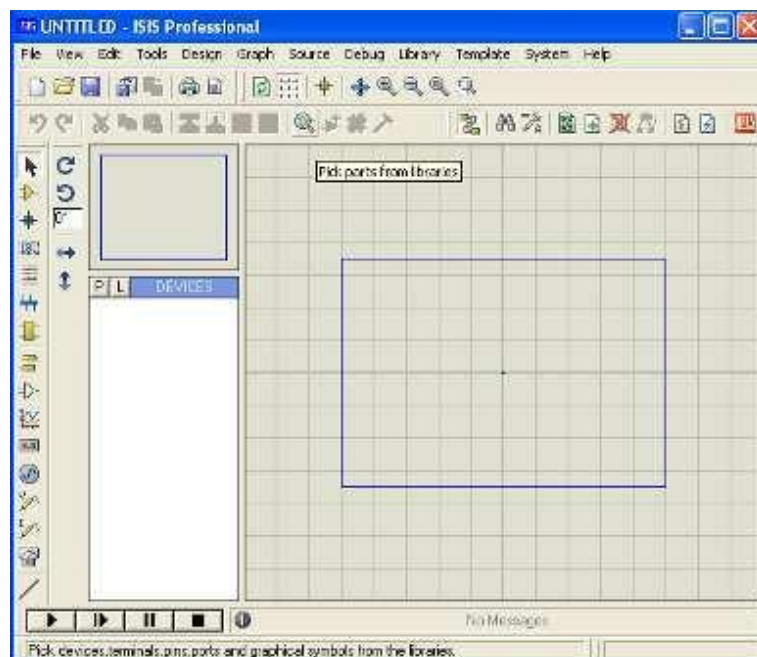
INPUT	OUTPUT
<i>A</i>	<i>B</i>
0	
1	

Part 2 - Proteus (Simulation Software)

Proteus has many features to generate both analog and digital results over a virtual environment. However, this lab will focus on tools that will be used in digital schematic designs and verification of basic logic gates.

Procedure

The Proteus software for simulation is installed in Digital Design Lab. Please follow the details below to figure out the usage of Proteus tools and process of simulation.

*Figure 1.8: Interface of Proteus software window*

Parts Browsing:

Proteus has many models of electronic equipment such as logic gates, many kinds of switches and basic electronic devices. The equipment can be placed by clicking on it and then a new window will pop-up as shown in Figure 1.12.

Finding Steps:

1. Type information of device such as “OR gate” in “*Keywords*’ box.
2. If some specific category is known, the device can narrow on focusing by selecting catalogue in the “*Category*” box.
3. After the information is entered, the list of related devices will appear in the “*Results*” window, so that needed device can be chosen and then click “OK” button to confirm selection in Figure 1.13.

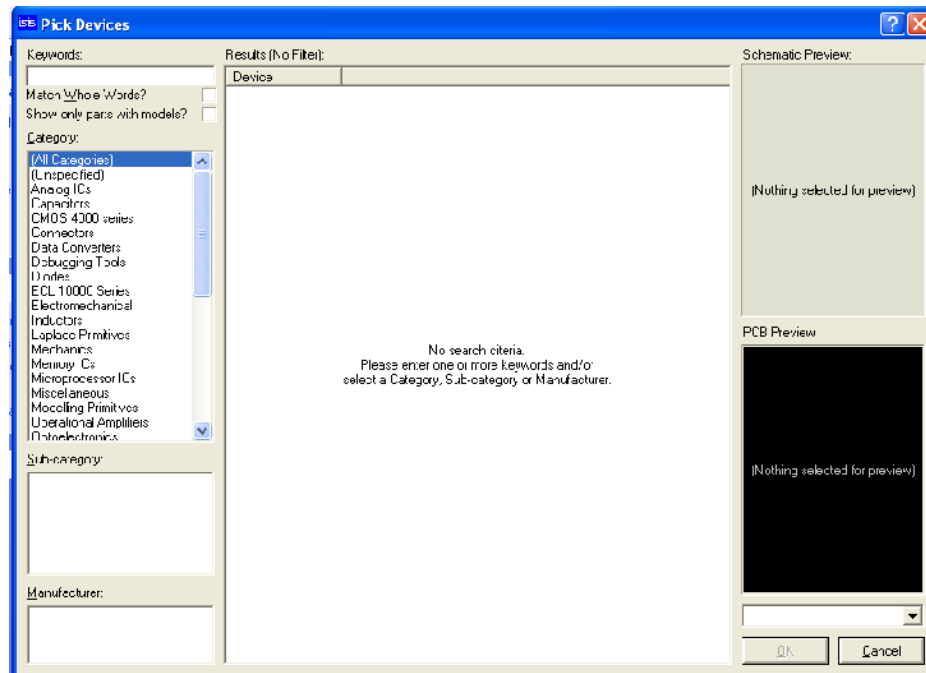


Figure 1.9: Pick Devices window in Proteus

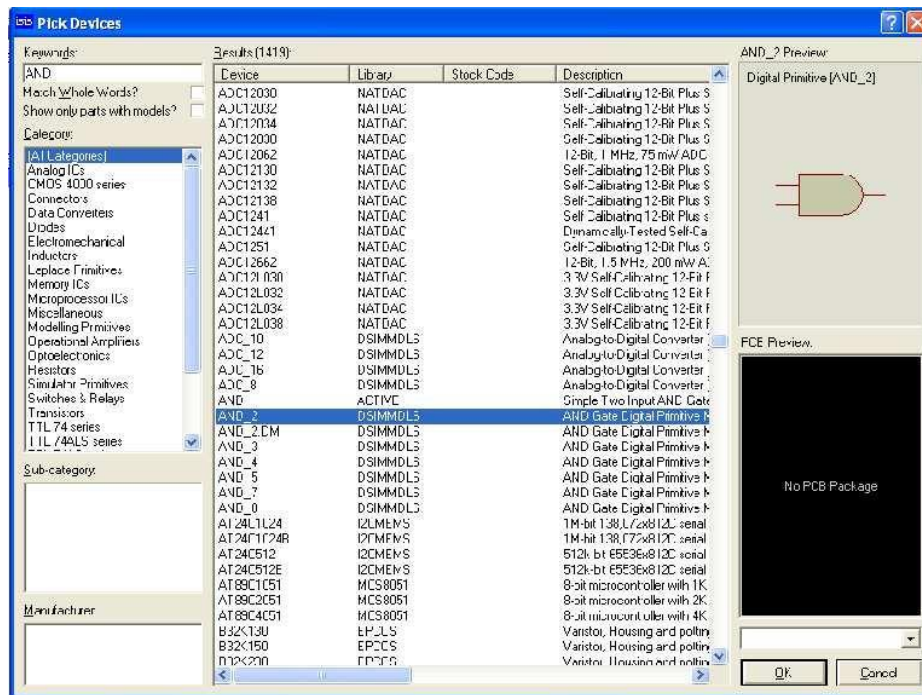


Figure 1.10: Pick selected Devices window in Proteus

Power supply and input signal Generator

All the electrical circuits require power supplies. The power supplies for logic circuits are represented in digital system design on Proteus because the schematic may be too complicated to understand for simulation section. Therefore, power supplies will be needed as input power for a system. Moreover, all the input generators, such as AC generator, DC and pulse, are contained in this category and it will be shown when clicked. In addition, “Ground” will not be available in this group. Because it is not an input signal it is just a terminal junction. Therefore, it will be grouped in the terminal category as shown in Figures 1.14 & 1.15.

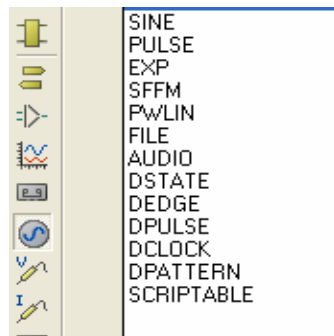


Figure 1.11: Power supplies window in Proteus

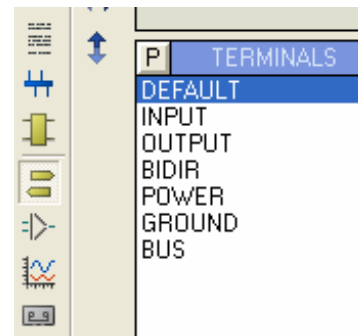


Figure 1.12: Terminals window in Proteus

Logic State:

In addition, there is another input that usually used in the digital circuit, but it does not exist in the real world as an equipment it is called as “**LOGIC STATE**”. It can be found in the picking part section (*type logic state and pick it as shown in Figure 1.16*).

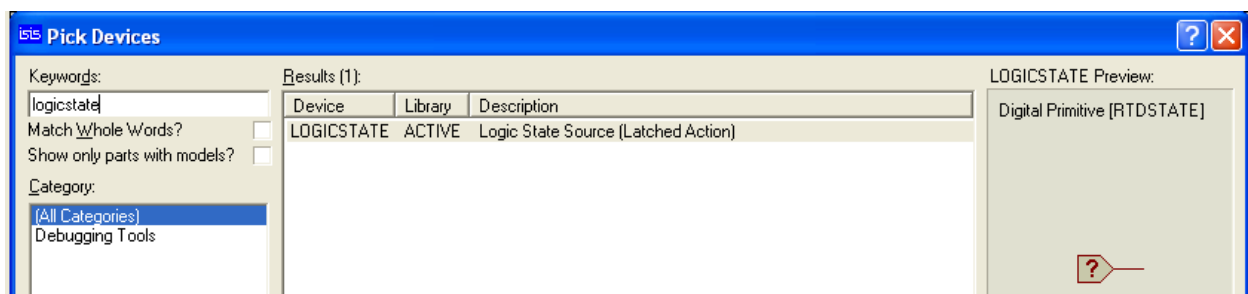


Figure 1.13: Logic State in Proteus

Placing Equipment:

Selecting all devices needed to be placed on the circuit window (*Gray window*) and make the required connections. It can be done by following steps:

1. Click on and select the first device that will be placed.
2. Place mouse wherever the device is preferred to place and then click the left button of the mouse. The device will be placed, if it is needed to be moved, click the right button of the mouse on the device symbol to select the mouse. Then hold this device with the left mouse button and move it to any desired place (Figure 1.17).

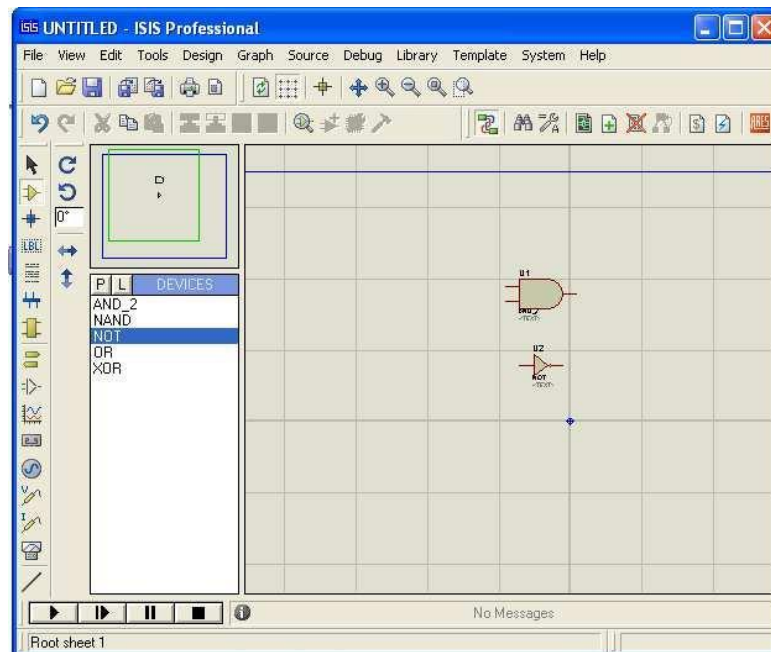


Figure 1.14: Placing the devices in Proteus

To make the connections between the devices, click on the source pin of a device and then move the cursor to destination pin of a device. In this step, the pink line will appear, and it will be a wire of the circuit after clicking the mouse on the destination pin of the circuit (as shown in Figure 1.18).

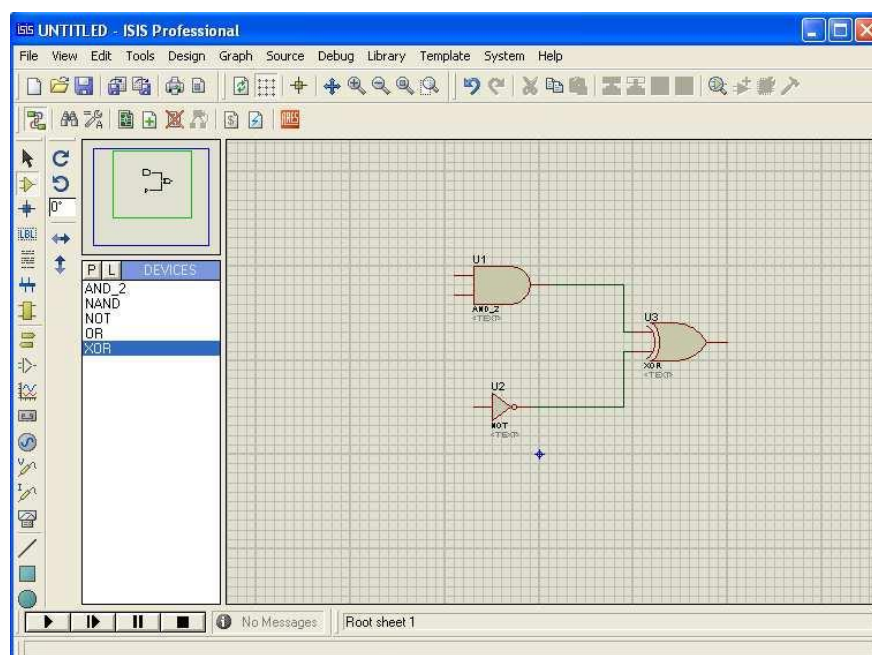


Figure 1.15: Making connection between devices to make a circuit

After wiring all devices and connect all inputs according to the circuit, the simulation is ready to run by clicking on Play button and stop button is used to stop the simulation.

3. Logic probe or LED can be used to observe the output state.

NOTE: The digital result on Proteus can be seen also in Small Square Box at the pin of the equipment & state can be shown in four colors. (Red= Logic 1, Blue = Logic 0, Gray= Unreadable and Yellow= Logic Congestion)

In-Lab Task 2:

Verify all the basic logic gates using the Proteus simulation tool and note down the values in the Tables 1.10 & 1.11 with the corresponding logic symbol and Boolean function. Then show the simulated logic circuit diagrams to your Lab Instructor.

Table 1.10: Observation Table for different gates

INPUTS		OUTPUTS					
<i>A</i>	<i>B</i>	<i>AND</i>	<i>OR</i>	<i>XOR</i>	<i>NAND</i>	<i>NOR</i>	<i>XNOR</i>
0	0						
0	1						
1	0						
1	1						

Table 1.11: Observation Table for NOT gate

INPUT	OUTPUT
<i>A</i>	<i>B</i>
0	
1	

Post-Lab Tasks:

Critical Analysis/Conclusion

Lab Assessment				
Pre-Lab			/1	/10
In-Lab			/5	
Post-Lab	Data Analysis	/4	/4	
	Data Presentation	/4		
	Writing Style	/4		
Instructor Signature and Comments				

LAB #02: Boolean Function Implementation using Universal Gates

Objectives

- This lab is designed to simulate and implement any logic function using universals gates (NAND/NOR).
- To build the understanding of how to construct any combinational logic function using NAND or NOR gates only.

Pre-Lab:

Background theory:

Digital circuits are more frequently constructed with universal gates. NAND and NOR gate are called universal gates. Any Boolean logic function can be implemented using NAND only or NOR only gates. NAND and NOR gates are easier to fabricate with electronic components than basic gates. Because of the prominence of universal gates in the design of digital circuits, rules and procedures have been developed for conversion from Boolean function given in terms of AND, OR, and NOT into its equivalent NAND and NOR logic diagram.

Read and understand the universal gates. List the truth tables of AND, OR, NOT, NAND, NOR and XOR gates. Identify the NAND and NOR ICs and their specification for CMOS and TTL families.

In lab:

This lab has two parts. In the first part, simulation and implementation of any logic expression by using only NAND gates are done. In the second part, the same procedure is done by using NOR gates only.

Part 1 - Implementing any logic expression by using only NAND gates

If we can show that the logical operations AND, OR, and NOT can be implemented with NAND gates, then it can be safely assumed that any Boolean function can be implemented with NAND gates.

Procedure

- Simulate NOT, AND, OR, XOR and XNOR gates in Proteus software, by using only NAND gates. Verify their truth tables.
- Insert the IC on the trainer's breadboard.
- Use any one or more of the NAND gates of the IC for this experiment.

- One or more Logic Switches of the trainer (S1 to S9) can be used for input to the NAND gate.
- For output indication, connect the output pin of the circuit to any one of the LEDs of the trainer (L0 to L15).

In-Lab Tasks-Part-1

In-Lab Task 1.1: Verification of NOT function

- Connect the circuit as shown in Figure 2.1.
- Connect +5V to pin 14 (Vcc) and Ground to pin 7 (GND) of the IC.
- By setting the switches to 1 and 0, verify that the output (F) of the circuit conforms to that of a NOT gate. Record your observations in the Table 2.1 below.

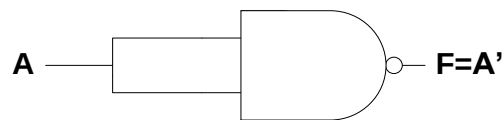


Figure 2.1: NOT gate using NAND gate

Table 2.1: Observation Table for NOT gate

INPUT	OUTPUT
<i>A</i>	<i>F</i>
0	
1	

In-Lab Task 1.2: Verification of AND function

- Connect the circuit as shown in Figure 2.2.
- Connect +5V to pin 14 (Vcc) and Ground to pin 7 (GND) of the IC.
- By setting the switches to 1 and 0, verify that the output (F) of the circuit conforms to that of an AND gate. Record your observations in Table 2.2 below.

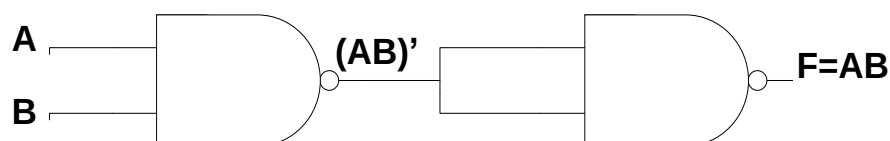


Figure 2.2: AND gate using NAND gates

Table 2.2: Observation Table for AND gate

INPUTS		OUTPUT
<i>A</i>	<i>B</i>	<i>F</i>
0	0	
0	1	
1	0	
1	1	

In-Lab Task 1.3: Verification of OR function

- Connect the circuit as shown in Figure 2.3.
- Connect +5V to pin 14 (Vcc) and Ground to pin 7 (GND) of the IC.
- By setting the switches to 1 and 0, verify that the output (F) of the circuit conforms to that of an OR gate. Record your observations in Table 2.3 below.

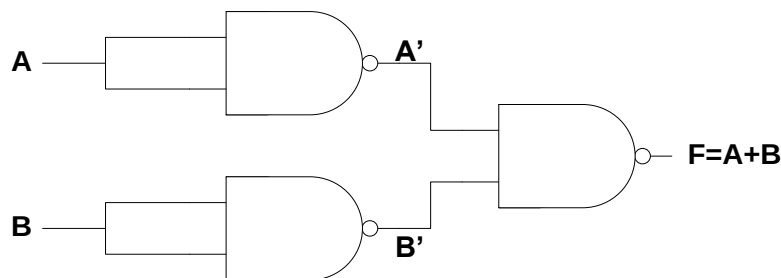


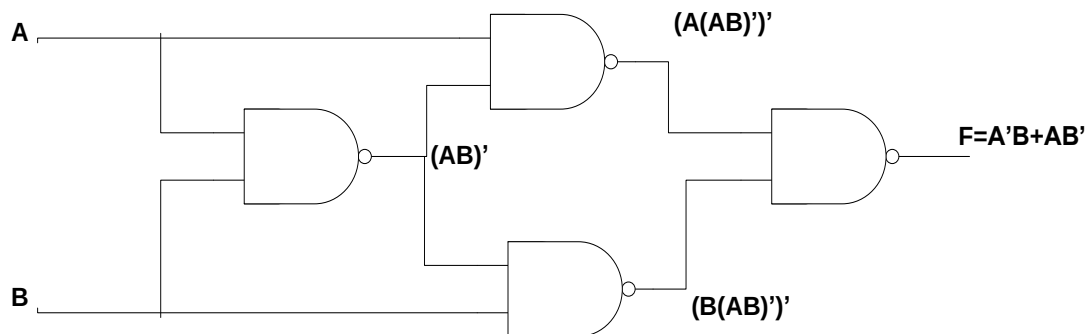
Figure 2.3: OR gate using NAND gates

Table 2.3: Observation Table for OR gate

INPUTS		OUTPUT
<i>A</i>	<i>B</i>	<i>F</i>
0	0	
0	1	
1	0	
1	1	

In-Lab Task 1.4: Verification of XOR function

- Connect the circuit as shown in Figure 2.4.
- Connect +5V to pin 14 (Vcc) and Ground to pin 7 (GND) of the IC.
- By setting the switches to 1 and 0, verify that the output (F) of the circuit conforms to that of an XOR gate. Record your observations in Table 2.4 below.

*Figure 2.4: XOR gate using NAND gates**Table 2.4: Observation Table for XOR gate*

INPUTS		OUTPUT
<i>A</i>	<i>B</i>	<i>F</i>
0	0	
0	1	
1	0	
1	1	

In-Lab Task 1.5: Verification of XNOR function

- Connect the circuit as shown in Figure 2.5.
- Connect +5V to pin 14 (Vcc) and Ground to pin 7 (GND) of the IC.
- By setting the switches to 1 and 0, verify that the output (F) of the circuit conforms to that of an XNOR gate. Record your observations in Table 2.5 below.

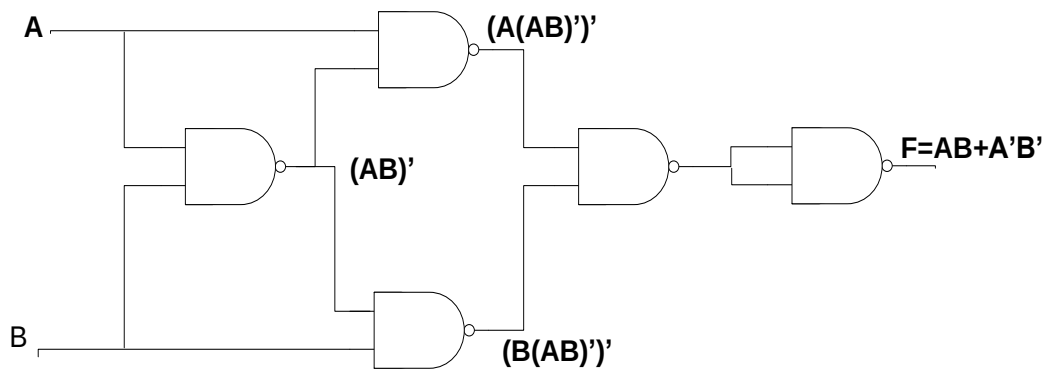


Figure 2.5: XNOR gate using NAND gates

Table 2.5: Observation Table for XNOR gate

INPUTS		OUTPUT
<i>A</i>	<i>B</i>	<i>F</i>
0	0	
0	1	
1	0	
1	1	

In-Lab Task 1.6: Implementation of any Boolean function (2-variables) using only NAND gates

$F(A, B) =$ _____

(Note: Boolean function will be specified by Lab Instructor)

Table 2.6: Observation Table for the given Boolean function

Inputs		Outputs	
<i>A</i>	<i>B</i>	Calculated <i>F_C</i>	Observed <i>F_O</i>
0	0		
0	1		
1	0		
1	1		

Part 2 - Implementing any logic expression by using only NOR gates

If we can show that the logical operations AND, OR, and NOT can be implemented with NOR gates, then it can be safely assumed that any Boolean function can be implemented with NOR gates.

Procedure

- Simulate NOT, AND and OR gates in Proteus software, by using only NOR gates. Verify their truth tables.
- Insert the IC on the trainer's breadboard.
- Use any one or more of the NOR gates of the IC for this experiment.
- One or more Logic Switches of the trainer (S1 to S9) can be used for input to the NOR gate.
- For output indication, connect the output pin of the circuit to any one of the LEDs of the trainer (L0 to L15).

In-Lab Tasks-Part-2

In-Lab Task 2.1: Verification of NOT function

- Connect the circuit as shown in Figure 2.6.
- Connect +5V to pin 14 (Vcc) and Ground to pin 7 (GND) of the IC.
- By setting the switches to 1 and 0, verify that the output (F) of the circuit conforms to that of an NOT gate. Record your observations in Table 2.7 below.



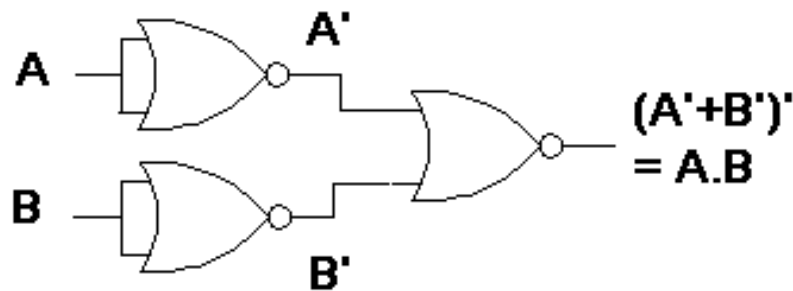
Figure 2.6: NOT gate using NOR gate

Table 2.7: Observation Table for NOT gate

INPUT	OUTPUT
<i>A</i>	<i>F</i>
0	
1	

In-Lab Task 2.2: Verification of AND function

- Connect the circuit as shown in Figure 2.7.
- Connect +5V to pin 14 (Vcc) and Ground to pin 7 (GND) of the IC.
- By setting the switches to 1 and 0, verify that the output (F) of the circuit conforms to that of an AND gate. Record your observations in Table 2.8 below.

*Figure 2.7: AND gate using NOR gates**Table 2.8: Observation Table for AND gate*

INPUTS		OUTPUT
<i>A</i>	<i>B</i>	<i>F</i>
0	0	
0	1	
1	0	
1	1	

In-Lab Task 2.3: Verification of OR function

- Connect the circuit as shown in Figure 2.8.
- Connect +5V to pin 14 (Vcc) and Ground to pin 7 (GND) of the IC.
- By setting the switches to 1 and 0, verify that the output (F) of the circuit conforms to that of an OR gate. Record your observations in Table 2.9 below.

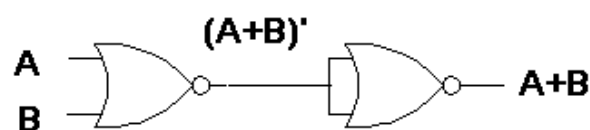
*Figure 2.8: OR gate using NOR gates*

Table 2.9: Observation Table for OR gate

INPUTS		OUTPUT
<i>A</i>	<i>B</i>	<i>F</i>
0	0	
0	1	
1	0	
1	1	

Post-Lab:

Simulate NAND, XOR and XNOR gates in Proteus software, by using only NOR gates. Verify their truth tables.

Critical Analysis/Conclusion

Lab Assessment				
Pre-Lab			/1	/10
In-Lab			/5	
Post-Lab	Data Analysis	/4	/4	
	Data Presentation	/4		
	Writing Style	/4		
Instructor Signature and Comments				

LAB #03: Design of Combinational Circuits and Logic Minimization Using Boolean Function

Objective

This Lab aims to enhance understanding of digital logic design principles, Boolean function minimization, and the practical application of software-based circuit simulation. Through this process, students will develop skills in logic circuit optimization, simulation, and verification.

Background Theory:

Combinational Circuits:

Combinational circuits are digital logic circuits where the output depends only on the current inputs and not on previous inputs (no memory elements). These circuits are composed of logic gates, and they perform operations like addition, subtraction, and logical operations.

Boolean Functions:

A Boolean function is a mathematical expression formed using binary variables, the constants 0 and 1, and the logic operations AND, OR, and NOT. Boolean functions can be simplified using algebraic methods or Karnaugh maps (K-maps).

Karnaugh Maps (K-Maps):

A K-map is a visual method of simplifying Boolean functions. It reduces the complexity of the Boolean expression, resulting in a simpler combinational circuit. The K-map for four variables is a 4x4 grid, representing all possible combinations of the variables.

Four-Variable Combinational Circuits:

In a four-variable combinational circuit, there are 16 possible input combinations (2^4). The circuit's complexity grows with the number of variables, making logic minimization crucial for practical implementation.

In-Lab:

Truth Table Construction:

Construct the truth table for a given four-variable Boolean function $F(A,B,C,D)$. Ensure all 16 combinations of inputs are listed along with their corresponding outputs.

Karnaugh Map Simplification:

1. Draw the K-map for the given Boolean function using the constructed truth table.
2. Group the adjacent ones (1s) in the K-map into the largest possible groups (1, 2, 4, 8, or 16 cells) to minimize the Boolean expression.

3. Derive the simplified Boolean expression from the grouped cells.

Logic Circuit Design:

1. Using the minimized Boolean expression, design the combinational circuit using basic logic gates (AND, OR, NOT).
2. Draw the logic circuit diagram representing the minimized expression.

Proteus Simulation:

1. Implement the designed combinational circuit using Proteus software.
2. Create a new project in Proteus and place the necessary logic gates according to the minimized Boolean expression.
3. Connect the inputs and outputs to logic probes or virtual LEDs to visualize the results.
4. Run the simulation and verify the circuit's operation by testing all possible input combinations and comparing the results with the truth table.

Optimization Verification:

1. Evaluate the complexity of the original and minimized Boolean expressions in terms of the number of logic gates used.
2. Confirm that the minimized circuit uses fewer gates and is more efficient.

Post-Lab:

Given a different four-variable Boolean function, repeat the steps of truth table construction, K-map simplification, and circuit design. Compare the efficiency of your design with the original circuit. Implement and verify the new design in Proteus as well.

Critical Analysis/Conclusion

Lab Assessment				
Pre-Lab			/1	/10
In-Lab			/5	
Post-Lab	Data Analysis	/4	/4	
	Data Presentation	/4		
	Writing Style	/4		
Instructor Signature and Comments				

LAB #04: Design and Implementation of Boolean Functions by Standard Forms

Objective

In this lab, we implement Boolean functions by using SoP (sums of product) and PoS (products of sum).

Equipment Required

KL-31001 Digital Logic Lab Trainer, Breadboard, Logic gate ICs (NAND & NOR).

Pre-Lab:

Boolean Algebra

Boolean algebra is algebra for the manipulation of objects that can take on only two values, typically true and false.

- It is common to interpret the digital value:
 - 0 as false
 - 1 as true

Boolean Function

A Boolean function typically has one or more input values and yields a result, based on these input value, in the range $\{0, 1\}$. A Boolean operator can be completely described using a table that lists inputs, all possible values for these inputs, and the resulting values of the operation.

A Boolean function can be represented in a truth table and it can be transformed from an algebraic expression into a circuit diagram composed of logic gates. In evaluating Boolean equations AND operation is performed before OR operation unless OR operation is enclosed with in brackets.

Basic Theorems

Table 4.1: Basic Boolean algebra theorems

Identity Name	AND form	OR form
Identity Law	$x \cdot 1 = x$	$x + 0 = x$
Null (or Dominance) Law	$x \cdot 0 = 0$	$x + 1 = 1$
Idempotent Law	$x \cdot x = x$	$x + 1 = x$
Inverse Law	$x \cdot \bar{x} = 0$	$x + \bar{x} = 1$
Commutative Law	$x \cdot y = y \cdot x$	$x + y = y + x$
Associative Law	$(x \cdot y) \cdot z = x \cdot (y \cdot z)$	$(x + y) + z = x + (y + z)$
Distributive Law	$x + (y \cdot z) = (x + y) \cdot (x + z)$	$x \cdot (y + z) = (x \cdot y) + (x \cdot z)$
Absorption Law	$x \cdot (x + y) = x$	$x + (x \cdot y) = x$
DeMorgan's Law	$\overline{(x \cdot y)} = \bar{x} + \bar{y}$	$\overline{(x + y)} = \bar{x} \cdot \bar{y}$
Double Complement Law	$\bar{\bar{x}} = x$	

Canonical Forms

Any Boolean function that is expressed as a sum of *minterms* or as a product of *maxterms* is said to be in its canonical form.

In general, the unique algebraic expression for any Boolean function can be obtained from its truth table by using an OR operator to combine all *minterms* for which the function is equal to 1.

Minterm

A *minterm* denoted as m_i , where $0 \leq i < 2^n$, is a product (AND) of the n variables (literals) in which each variable is complemented if the value assigned to it is 0, and uncomplemented if it is 1.

- 1-minterms = minterms for which the function $F = 1$.
- 0-minterms = minterms for which the function $F = 0$.
- Any Boolean function can be expressed as a sum (OR) of its 1-minterms.

A shorthand notation:

$$F(\text{list of variables}) = \sum (\text{list of 1-minterm indices})$$

- The inverse of the function can be expressed as a sum (OR) of its 0-minterms.

A shorthand notation:

$$\bar{F}(\text{list of variables}) = \sum (\text{list of 0-minterm indices})$$

Example:

Table 4.2: 3-variables Minterm example

x	y	z	Minterms	F	$\bar{F}$
0	0	0	$m_0 = x'y'z'$	0	1
0	0	1	$m_1 = x'y'z$	0	1
0	1	0	$m_2 = x'yz'$	0	1
0	1	1	$m_3 = x'yz$	1	0
1	0	0	$m_4 = xy'z'$	0	1
1	0	1	$m_5 = xy'z$	1	0
1	1	0	$m_6 = xyz'$	1	0
1	1	1	$m_7 = xyz$	1	0

$$F = x'yz + xy'z + xyz' + xyz = m_3 + m_5 + m_6 + m_7$$

or

$$F(x, y, z) = \sum (3, 5, 6, 7)$$

Similarly,

$$\bar{F} = x'y'z' + x'y'z + x'yz' + xy'z' = m_0 + m_1 + m_2 + m_4$$

or

$$\bar{F}(x, y, z) = \sum (0, 1, 2, 4)$$

Maxterm

A *maxterm* denoted as M_i , where $0 \leq i < 2n$, is a sum (OR) of the n variables (literals) in which each variable is complemented if the value assigned to it is 1, and un complemented if it is 0.

- 0-maxterms = maxterms for which the function $F = 0$.
- 1-maxterms = maxterms for which the function $F = 1$.
- Any Boolean function can be expressed as a product (AND) of its 0-maxterms.

A shorthand notation:

$$F(\text{list of variables}) = \prod (\text{list of 0-maxterm indices})$$

- The inverse of the function can be expressed as a product (AND) of its 1-maxterms.

A shorthand notation:

$$\bar{F}(\text{list of variables}) = \prod (\text{list of 1-maxterm indices})$$

Example:

Table 4.3: 3-variables Maxterm example

x	y	z	Maxterms	F	$\bar{F}$
0	0	0	$M_0 = x + y + z$	0	1
0	0	1	$M_1 = x + y + z'$	0	1
0	1	0	$M_2 = x + y' + z$	0	1
0	1	1	$M_3 = x + y' + z'$	1	0
1	0	0	$M_4 = x' + y + z$	0	1
1	0	1	$M_5 = x' + y + z'$	1	0
1	1	0	$M_6 = x' + y' + z$	1	0
1	1	1	$M_7 = x' + y' + z'$	1	0

$$F = (x + y + z). (x + y + z'). (x + y' + z). (x' + y + z) = M_0 . M_1 . M_2 . M_4$$

or

$$F(x, y, z) = \prod (0, 1, 2, 4)$$

Similarly,

$$\bar{F} = (x + y' + z'). (x' + y + z'). (x' + y' + z). (x' + y' + z') = M_3 + M_5 + M_6 + M_7$$

or

$$\bar{F}(x, y, z) = \prod (3, 5, 6, 7)$$

Standard Forms

SoP (Sum of Products):

The term "**Sum of Products**" or "**SoP**" is widely used for the canonical form that is a disjunction (OR) of *minterms*.

PoS (Product of Sum):

The term "**Product of Sums**" or "**PoS**" for the canonical form that is a conjunction (AND) of *maxterms*.

Pre-Lab Tasks:

1. Express the Boolean function $F = x + yz$ as a sum of *minterms* by using truth table.
2. Express $F' = (x + yz)'$ as a product of *maxterms*.
3. Given the function as defined in the truth table (Table 4.4), express F using sum of *minterms* and product of *maxterms*.

Table 4.4: Truth Table for F (Pre-Lab Task 3)

x	y	z	Minterms	Maxterms	F	$\bar{F}$
0	0	0	$m_0 = x'y'z'$	$M_0 = x + y + z$	0	1
0	0	1	$m_1 = x'y'z$	$M_1 = x + y + z'$	1	0
0	1	0	$m_2 = x'yz'$	$M_2 = x + y' + z$	0	1
0	1	1	$m_3 = x'yz$	$M_3 = x + y' + z'$	1	0
1	0	0	$m_4 = xy'z'$	$M_4 = x' + y + z$	1	0
1	0	1	$m_5 = xy'z$	$M_5 = x' + y + z'$	0	1
1	1	0	$m_6 = xyz'$	$M_6 = x' + y' + z$	0	1
1	1	1	$m_7 = xyz$	$M_7 = x' + y' + z'$	0	1

In-Lab Tasks:

Circuit Implementation using Proteus

1. First, make the circuit diagram of the given Task.
2. Select appropriate logic gate ICs which are needed.
3. Make connections according to the circuit diagram you made.
4. Connect the input to data switches and output to the logic indicator.
5. Follow the input sequence and record the output.

TASK: Implement the circuit for the given function “ F ”. Function’s output is given in Table 4.5. Find its Boolean expression in SoP and PoS forms.

Table 4.5: Truth Table for F (In-Lab Task)

A	B	C	D	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Boolean Equations:

Sum of Min-terms equation of F: _____

Reduced SOP form equation of F: _____

Product of Max-terms equation of F: _____

Reduced POS form equation of F: _____

Reduced form calculation:

Post-Lab Tasks:

1. Write a Verilog code for the sum of *minterms* circuit, F_1 , (Structural Level).
2. Write a Verilog code for the reduced SoP circuit, F_2 , (Structural Level).
3. Write a Verilog code for the reduced PoS circuit F_3 , (Structural Level).
4. Simulate and verify the outputs by making an appropriate stimulus for the above modules.

Critical Analysis/Conclusion

Lab Assessment				
Pre-Lab			/1	/10
In-Lab			/5	
Post-Lab	Data Analysis	/4	/4	
	Data Presentation	/4		
	Writing Style	/4		
Instructor Signature and Comments				

LAB #05: Logic Minimization of Complex Functions using Automated Tools

Objective

In this lab, students will learn Karnaugh Map minimization and how to use logic minimization automated tools for an excessive number of variables in a function.

Pre-Lab:

Background Theory:

We can simplify the equations and find a Boolean function of any truth table using K-Map.

K-Maps:

- 3-Variables:

		B					
		BC		00	01	11	10
A	0	m_0	m_1	m_3	m_2		
	1	m_4	m_5	m_7	m_6		

C

- 4-Variables:

		C				
		CD		00	01	11
AB	00	m_0	m_1	m_3	m_2	
	01	m_4	m_5	m_7	m_6	

B

A

University Islamabad

11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

D

In-Lab:

Part 1: Automated Tool (Karnaugh Map Minimizer) (for variable >4)

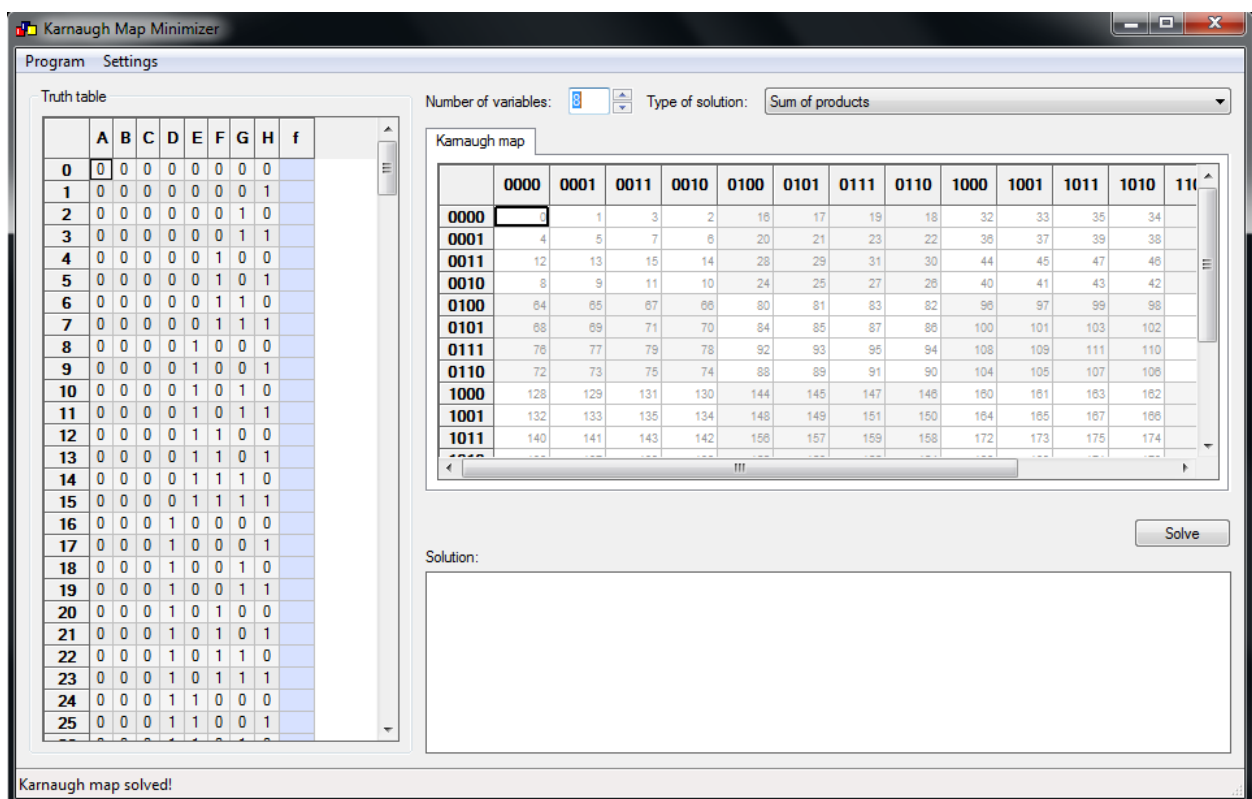


Figure 5.1: Karnaugh Map Minimizer Automated Tool Interface

Procedure:

1. First, make the circuit diagram for the desired task or subtask given by the Lab instructor.
2. Using k-map minimizer tool derive the simplified function:
 - a. Set the number of variables
 - b. Set the type of solution
 - c. Set the values of function 'F'. (Function values can be generated randomly) *

- d. Click on solve button then the solution will come on solution screen.
3. Implement the circuit using logic gate (which is/are needed) for the tasks.
4. Make connections according to the circuit diagram you made.
5. Connect the inputs to data switches and output to the logic indicator.
6. Follow the input sequence and record the outputs in Table 5.1.

In-Lab Task 1:

Implement the minimized function given below using logic gate IC(s).

$$F(A, B, C, D) = \sum (\quad , \quad , \quad , \quad)$$

(Note: minterms will be specified by Lab Instructor)

Function in sum of Min-terms form: _____

Function in a simplified form using K-map: _____

Simplified calculation (K-map):

Circuit Diagram of a min-terms form of the Function:

Number of gates used: _____

Circuit Diagram of a simplified Function:

Number of gates used: _____

Truth Table:*Table 5.1: Observation Table for In-Lab Task*

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>F</i>	Observed Outputs	
					<i>F</i> ₁	<i>F</i> ₂
0	0	0	0			
0	0	0	1			
0	0	1	0			
0	0	1	1			
0	1	0	0			
0	1	0	1			
0	1	1	0			
0	1	1	1			
1	0	0	0			
1	0	0	1			
1	0	1	0			
1	0	1	1			
1	1	0	0			
1	1	0	1			
1	1	1	0			
1	1	1	1			

*F*₁: Output of sum of Min-terms form circuit.

*F*₂: Output of simplified function circuit.

Post-Lab Tasks:

Select a different complex Boolean function with more variables (e.g., five or six variables). Use the automated tool to perform logic minimization on this function. Implement and simulate the minimized circuit in Proteus or another digital simulation software. Verify the functionality of the circuit and document the process, highlighting the differences in complexity between the original and minimized circuits

Critical Analysis/Conclusion

Lab Assessment				
Pre-Lab			/1	/10
In-Lab			/5	
Post-Lab	Data Analysis	/4	/4	
	Data Presentation	/4		
	Writing Style	/4		

Instructor Signature and Comments

LAB #06: Design 4-bit Binary to Gray Code Converter

Objective

Understand the principles of binary to gray code conversion and its applications. Develop skills in digital circuit design and simulation, reinforcing theoretical knowledge through practical application.

Pre-Lab:

Background Theory:

In the coding, when numbers, letters or words are represented by a specific group of symbols, it is said that the number, letter or word is being encoded. The group of symbols is called as a code. The digital data is represented, stored and transmitted as group of binary bits. This group is also called as binary code. The binary code is represented by the number as well as alphanumeric letter e.g., Binary Coded Decimal (BCD), 2421 code, Excess-3 and Gray code.

Gray Code

It is the non-weighted code and it is not arithmetic code. That means there are no specific weights assigned to the bit position. It has a very special feature that, only one bit will change each time the decimal number is incremented as shown in Table 6.1. As only one bit changes at a time, the gray code is called as a unit distance code. The gray code is a cyclic code. Gray code cannot be used for arithmetic operation.

The Gray code is used in many applications in which the normal sequence of binary numbers may produce an ambiguity or error during the transition from one number to the next. For example, from 011 to 100 (which bit will change first?) may produce an intermediate erroneous number 101 if the value of least significant bit takes longer time to change than the other two bits value change. As only one bit changes its value during any transition between two number in Gray code, therefore, this problem is eliminated by the usage of Gray code.

Gray code is a binary numeral system where two successive values differ in only one bit. This property is useful in minimizing errors in digital circuits and in applications such as error correction in digital communication systems.

A 4-bit binary to Gray code converter takes a 4-bit binary number as input and outputs the corresponding 4-bit Gray code. The conversion between binary and Gray code is defined as follows:

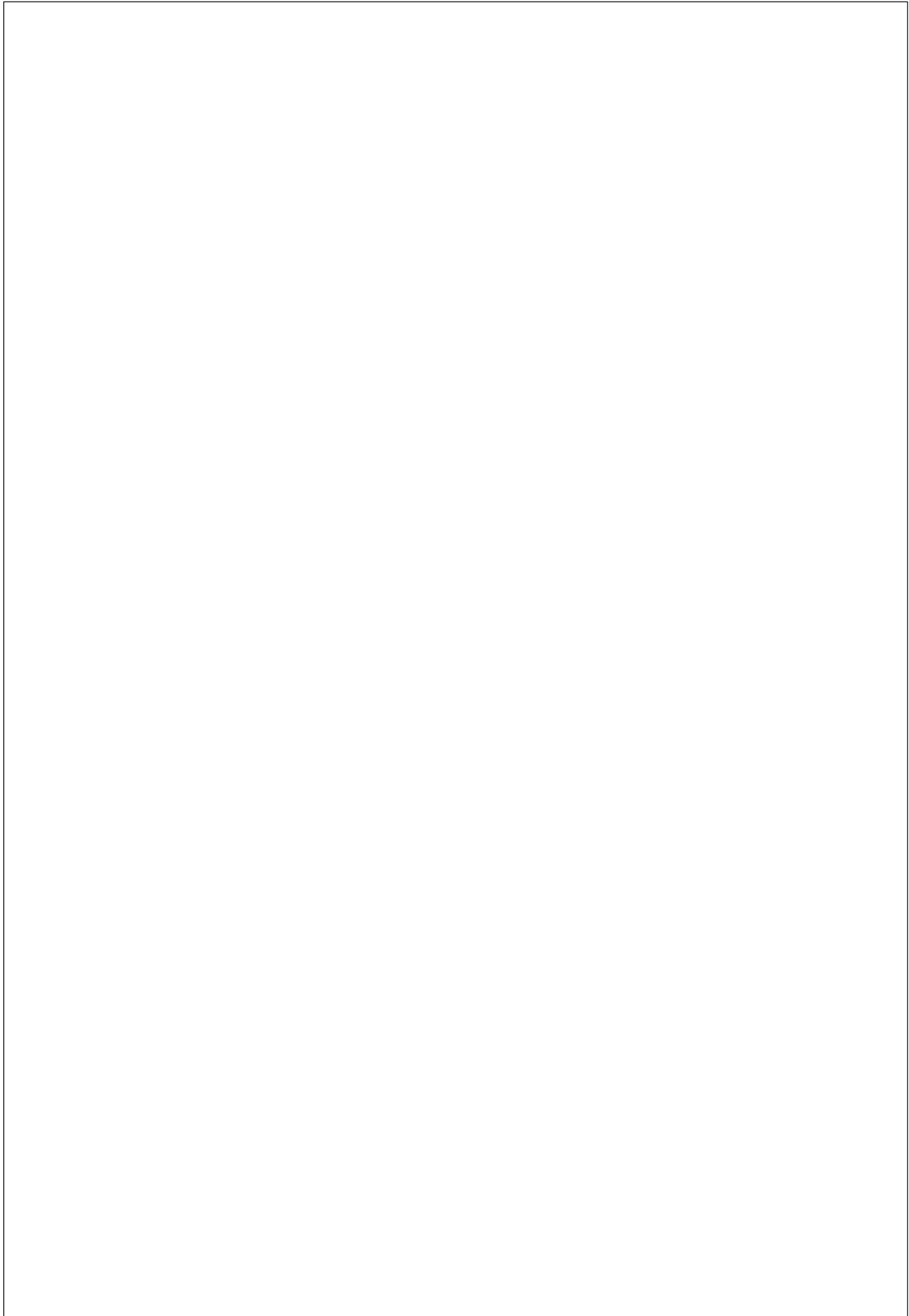
The most significant bit (MSB) of the Gray code is the same as the MSB of the binary code.

LAB #06: Design 4-bit Binary to Gray Code Converter

Each subsequent bit of the Gray code is found by performing an XOR operation between the current binary bit and the previous binary bit.

K-map and Boolean expressions Calculation:

Gate-Level Circuit Diagram of a simplified Function:



In-Lab:

Task 1: Draw a Truth Table

Construct the truth table for the 4-bit binary to Gray code converter. List all possible 4-bit binary inputs (0000 to 1111). Determine the corresponding 4-bit Gray code outputs using the conversion rules.

Task 2: Implementation on Proteus

Create a New Project:

- Open Proteus and create a new project for the 4-bit binary to Gray code converter.

Place Components:

- Place four input pins (representing the 4-bit binary input).
- Place four output pins (representing the 4-bit Gray code output).
- Place the necessary logic gates (XOR gates) according to the conversion logic:
 - $G3 = B3$ (MSB remains the same)
 - $G2 = B3 \text{ XOR } B2$
 - $G1 = B2 \text{ XOR } B1$
 - $G0 = B1 \text{ XOR } B0$ (LSB)

Connect the Circuit:

- Connect the input pins to the respective logic gates to implement the conversion logic.
- Connect the outputs of the logic gates to the output pins.

Simulation:

- Run the simulation in Proteus.
- Apply all possible 4-bit binary inputs (0000 to 1111) and verify the corresponding 4-bit Gray code outputs.
- Observe the outputs and ensure they match the truth table.

Documentation:

- Capture screenshots of the Proteus circuit and the simulation results.
- Document the steps taken in Proteus to design and simulate the 4-bit binary to Gray code converter.

Post-Lab Task:

1. Analyse the circuits implementation of In-lab Task 1 and In-lab Task 2 provide detail report including screenshots of simulation and its result.

Critical Analysis/Conclusion

Lab Assessment				
Pre-Lab			/1	/10
In-Lab			/5	
Post-Lab	Data Analysis	/4	/4	
	Data Presentation	/4		
	Writing Style	/4		
Instructor Signature and Comments				

LAB #07: Design and Implementation of $n - bit$ Adder/Subtractor

Objectives

This experiment is to design gate-level hierarchal description of a 4 – *bit* binary Adder / Subtractor in proteus simulation.

Pre-Lab:

Background theory:

Binary Adder is digital circuit that produces the arithmetic sum of two binary numbers. It can be constructed with full adders connected in cascade, with the output carry from each full adder connected to the input carry of the next full adder in the chain. The process of addition proceeds on a bit-by-bit basis, right to left, beginning with the least significant bit. After the least significant bit, addition at each position adds not only the respective bits of the words, but also consider a possible carry bit from addition at the previous position. Addition of $n - bit$ binary numbers requires the use of a n full adder, or a chain of one-half adder and $n - 1$ full adders. The four-bit adder is an example of a standard component. It can be used in many applications involving arithmetic operations.

Binary Adder/Subtractor can be combined into one circuit with one common binary adder by including an exclusive-OR gate with each full adder. A four-bit adder/subtractor circuit of two binary numbers $A(A_3A_2A_1A_0)$ and $B(B_3B_2B_1B_0)$ is shown in Figure 7.1. The mode input M controls the operation. When $M = 0$, the circuit is an adder. As $B \oplus 0 = B$, $C_0 = 0$, full adders receive the value of B , and the circuit performs $A + B$. when $M = 1$, the circuit becomes a subtractor. Now we have $B \oplus 1 = \bar{B}$ and $C_0 = 1$. The B input is complemented and a 1 is added through the input carry (C_0). The circuit performs the operation A plus the 2's complement of B i.e., $A - B$. The exclusive-OR with output V is for detecting an overflow and C is carry out.

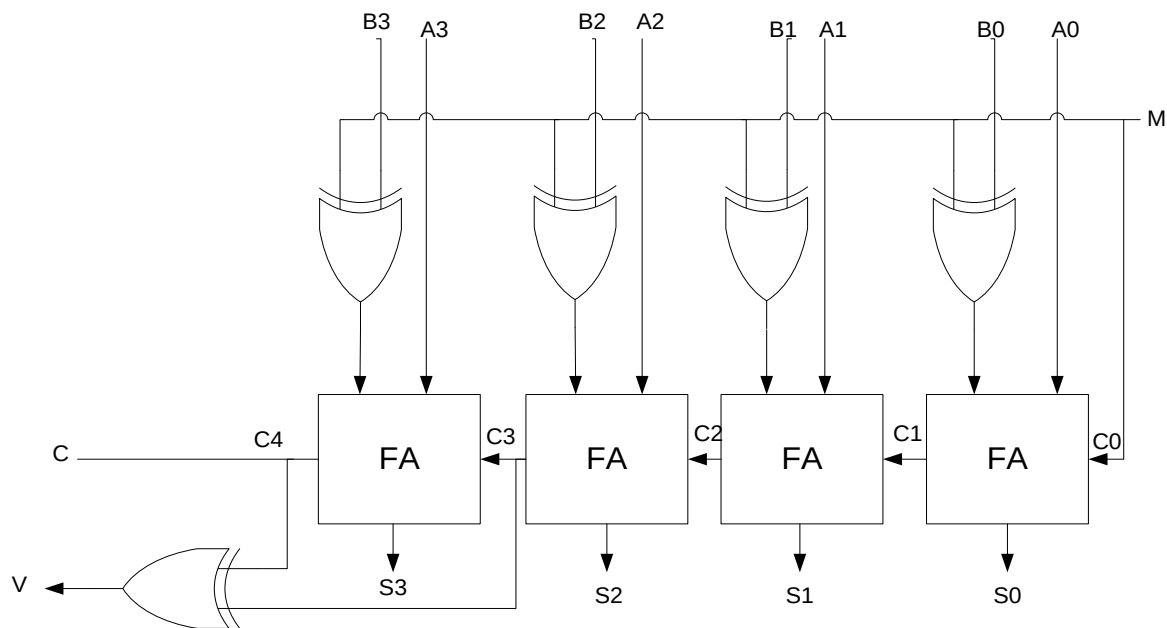


Figure 7.1: Binary adder/subtractor with overflow

In-Lab Tasks:

Part 1 (a) – Implementation of a 4-bit binary Adder / Subtractor without overflow using Logic gates

Procedure

- Implement a 4 – bit binary adder / Subtractor circuitry as shown in Figure 7.1, without overflow detection.
- Use 4-bit Full Adder (pin description is shown in Figure 7.2) and XOR gates.

16	15	14	13	12	11	10	9
B_3	S_3	C_4	C_0	Gnd	B_0	A_0	S_0
7483 (4-bit Full Adder)							
A_3	S_2	A_2	B_2	Vcc	S_1	B_1	A_1
1	2	3	4	5	6	7	8

Figure 7.2: IC 7483 (4-bit Full Adder) pin configuration

- Connect all the inputs M , A and B (4-bits for each binary number) on slide switches, and the outputs are observed on LEDs.
- Observe the behaviour of Adder /Subtractor by choosing different binary numbers and fill the observation Table given below.

Table 7.1: Observation table for Adder/Subtractor circuit (Implemented using Proteus)

		$V_{\text{Calculated}}$	C_{out}	<i>Bit</i>				M
				3	2	1	0	
1	A							
	B							
	Sum							
	$Difference$							
2	A							
	B							
	Sum							
	$Difference$							
3	A							
	B							
	Sum							
	$Difference$							
4	A							
	B							
	Sum							
	$Difference$							

Part 1 (b) - Implementing hierarchal description of a 4-bit binary Adder / Subtractor

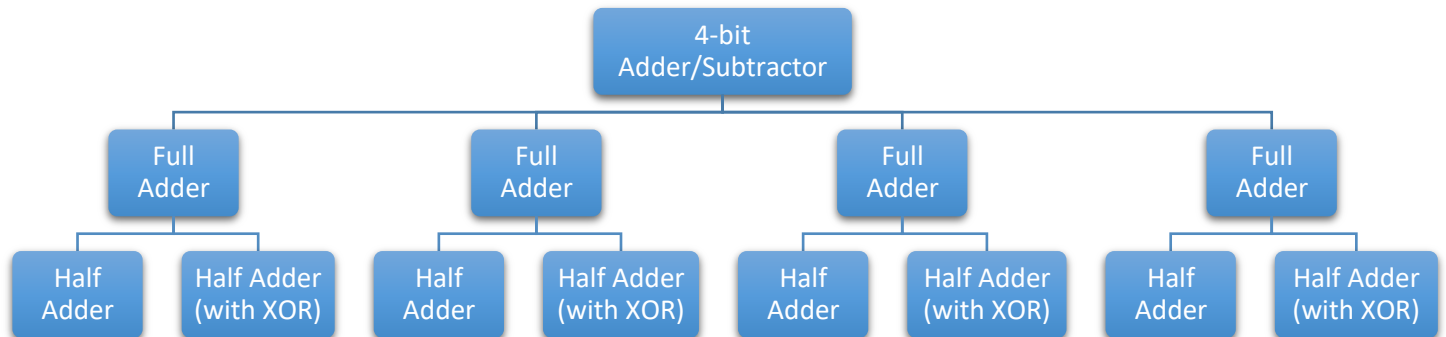


Figure 7.3: Hierarchal description of a 4-bit binary Adder / Subtractor

Procedure

- Interconnect four full adder modules to design a 4-bit adder/subtractor.
- Add XOR gates to control the subtraction operation.
- Include an additional XOR gate for overflow detection.
- Use slide switches to represent the 4-bit binary inputs AAA and BBB.
- Use a push switch to represent the operation mode input MMM (0 for addition, 1 for subtraction).
- Connect LEDs to observe the outputs of the 4-bit adder/subtractor.
- Use the Proteus interface to place and wire the components accordingly.
- Simulate the design in Proteus.
- Test the behaviour of the adder/subtractor by applying different binary numbers as inputs.
- Observe the results on the LEDs.
- Record the results in an observation on Table 7.2.

Table 7.2: Observation table for Adder/Subtractor circuit

		V	C_{out}	Bit				M
				3	2	1	0	
→	A							

	<i>B</i>							
	<i>Sum</i>							
	<i>Difference</i>							
2	<i>A</i>							
	<i>B</i>							
	<i>Sum</i>							
	<i>Difference</i>							
3	<i>A</i>							
	<i>B</i>							
	<i>Sum</i>							
	<i>Difference</i>							
4	<i>A</i>							
	<i>B</i>							
	<i>Sum</i>							
	<i>Difference</i>							

Post-Lab:

- Using proteus, write implement an $n - bit$ Adder /Subtractor:
 - Make one stimulus for two different parameter values and show the results.
- Analyse the circuits implementation of Structural-Level 4 – bit Adder/Subtractor and Behavioral-Level 4 – bit Adder/Subtractor, in terms of resource utilization.

HDL codes / Results (Wave Forms) / Analysis

Critical Analysis/Conclusion

Lab Assessment				
Pre-Lab			/1	/10
In-Lab			/5	
Post-Lab	Data Analysis	/4	/4	
	Data Presentation	/4		
	Writing Style	/4		
Instructor Signature and Comments				

LAB #08: Design and Implementation of $n - bit$ Binary Multiplier

Objective

In this lab, we will seek how to design $n - bit$ binary multiplier and its implementation on proteus.

Pre-Lab:

Background Theory:

Binary Multiplier

Binary multiplier is used to multiply two binary numbers. Multiplication of binary numbers is performed in the same way as multiplication of decimal numbers. It is build using binary adders. The common multiplication method is “add and shift” algorithm. If the multiplicand is $N - bits$ and the Multiplier is $M - bits$ then there is $N \times M$ partial product. AND gates are used to generate the Partial Products. Note that in binary multiplication, the processes involve shifting the multiplicand, and adding the shifted multiplicand or zero. Each bit of the multiplier determines whether a 0 is added or a shifter version of the multiplicand (0 implies zero added, 1 implies shifted multiplicand added). Thus, we can infer a basic shift-and-add algorithm to implement unsigned binary multiplication.

General Expression

$$\begin{array}{r}
 \begin{array}{cc}
 & B_1 & B_0 \\
 \times & A_1 & A_0 \\
 \hline
 & A_0 B_1 & A_0 B_0 \\
 A_1 B_1 & A_1 B_0 & \\
 \hline
 C_3 & C_2 & C_1 & C_0
 \end{array}
 \end{array}$$

Page 64

In-Lab Task 1: Implement 4 – bit by 2 – bit binary multiplier

- Implement 4 – bit by 2 – bit binary multiplier using Proteus as shown in Figure 8.2.
- Use 4 – bit Full Adder block and AND gate.
- Observe the behaviour of a binary Multiplier by choosing different binary numbers and verify the results.

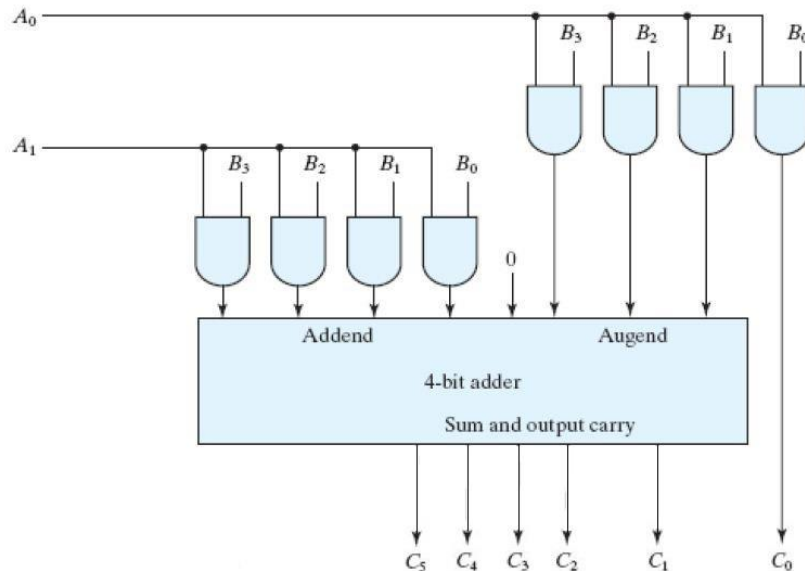


Figure 8.2: 4 – bit by 2 – bit binary multiplier

In-Lab Task 2: Implement 4 – bit by 3 – bit binary multiplier

1. Using 4 – bit by 3 – bit multiplier block.
 - a. Use 4 – bit Adder module made in Lab7.
2. Site all the inputs A and B on slide switches and the outputs are observed on LEDs.

Post-Lab Tasks:

1. Design parameterized Multiplier:
 - Make single circuit for two different parameter values and show the results.
2. Analyse the circuits implementation and briefly explain its working.

Critical Analysis/Conclusion

Lab Assessment				
Pre-Lab			/1	/10
In-Lab			/5	
Post-Lab	Data Analysis	/4	/4	
	Data Presentation	/4		
	Writing Style	/4		
Instructor Signature and Comments				

LAB #09: Design and Implementation of BCD to 7-Segment Decoder

Objective

In this lab, we design and implement BCD to 7-Segment decoder.

Pre-Lab:

Background Theory:

Binary to BCD Decoder

In digital, **binary-coded decimal (BCD)** is a class of binary encodings of decimal numbers where each decimal digit is represented by a fixed number of bits, usually four. Table 9.1 gives the four-bit code for one decimal digit. A number with k decimal digits will require $4k$ bits in BCD.

Table 9.1: Binary coded decimal (BCD)

Decimal Value	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

BCD to 7-Segment Decoder

BCD to 7-segment decoder decodes the BCD value on 7-Segment display.

7-Segment Display

A seven-segment display (SSD) is a form of the electronic display device for displaying decimal numerals. The seven elements of the display can be selected in different combinations to represent the decimal numerals. Often the seven segments are arranged in an oblique (slanted) arrangement, which aids readability. Seven-segment displays may use a light-emitting diode (LED) or a liquid crystal display (LCD), for each segment, or other light-generating or controlling techniques. There are two types of simple LED package 7-Segment display:

- Common Anode
- Common Cathode

In-Lab Task : Design and implementation of a BCD to 7-Segment decoder on FPGA

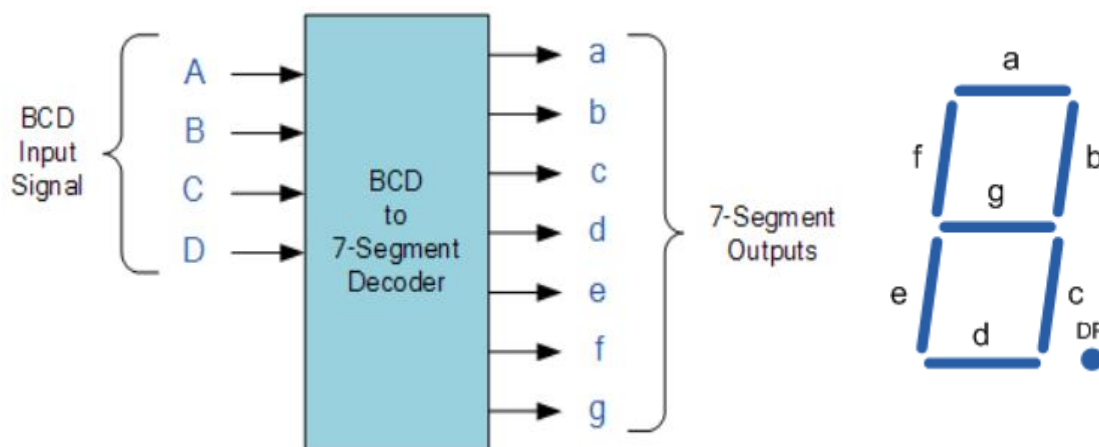


Figure 9.1: BCD to 7-Segment decoder module

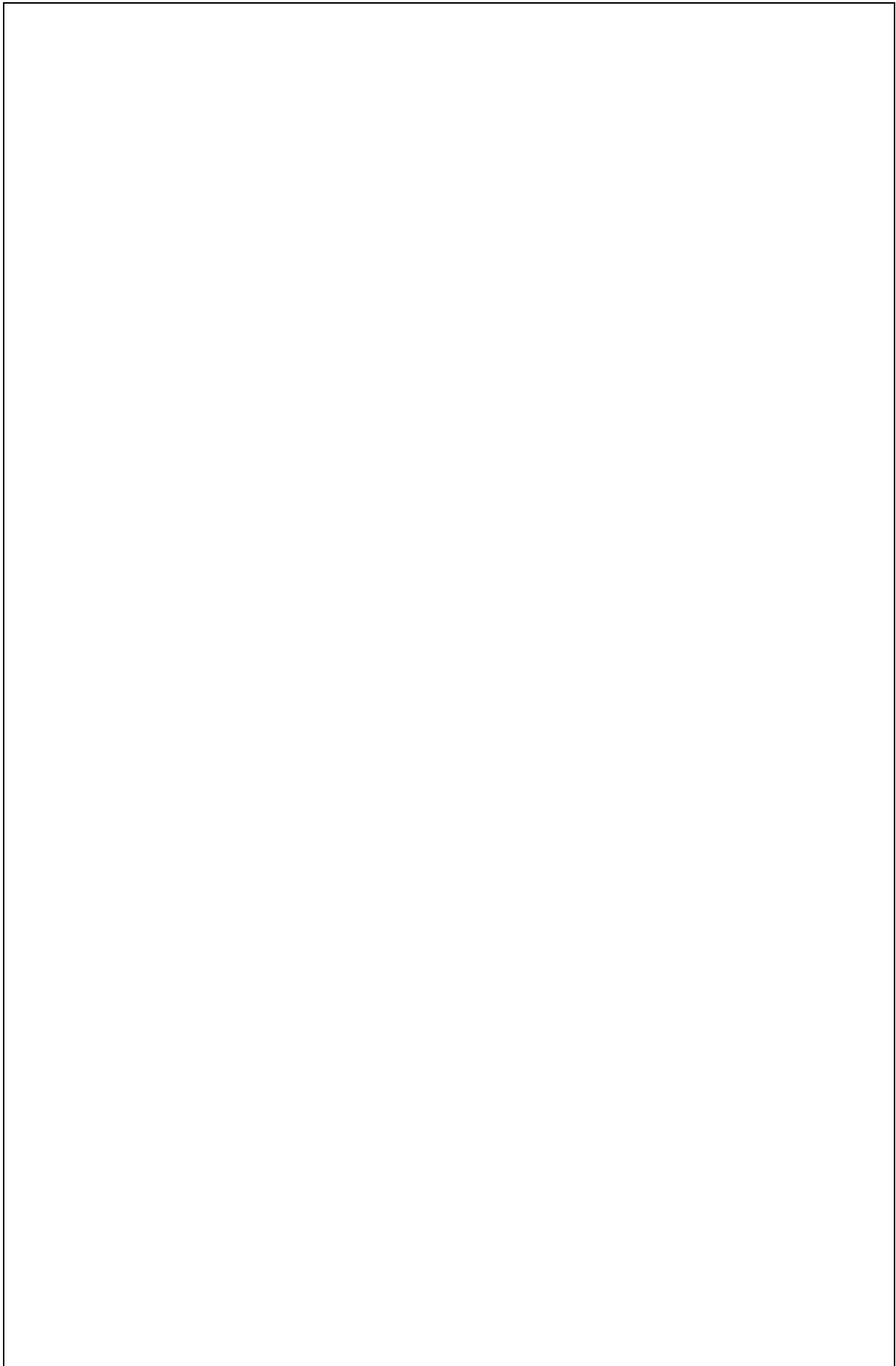
- First, make a truth table for BCD to 7-Segment decoder which has 4-inputs (BCD) and 8-outputs (A to G and DP) of the 7-Segment display (*Fill Table 9.2*).
- Using Behavioral model, write a Verilog description of BCD to 7-Segment decoder using Case statements.
- Implement it on Proteus.

Table 9.2: Truth table for BCD to 7-Segment decoder

BCD				7-Segment								
<i>D</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>DP</i>	<i>Hex</i>
0	0	0	0									
0	0	0	1									
0	0	1	0									
0	0	1	1									
0	1	0	0									
0	1	0	1									
0	1	1	0									
0	1	1	1									
1	0	0	0									
1	0	0	1									
Decimal Point (DP)												
Error (E)												

Post-Lab Task:

Modify the BCD to 7-segment decoder circuit to handle invalid BCD inputs (1010 to 1111). Implement a mechanism to turn off the 7-segment display or display a specific pattern (e.g., all segments off) for invalid inputs. Simulate the modified circuit in Proteus and verify the error handling functionality.



Critical Analysis/Conclusion

Lab Assessment				
Pre-Lab			/1	/10
In-Lab			/5	
Post-Lab	Data Analysis	/4	/4	
	Data Presentation	/4		
	Writing Style	/4		
Instructor Signature and Comments				

Lab #10: Design and Implementation of a Sequence Detector using Mealy/Moore Machine

Objective

This experiment is to design a Sequence detector using Mealy/Moore models of Finite State Machine (FSM) and implement the design on Proteus.

Pre-Lab:

Background theory:

The most general model of a sequential circuit has inputs, outputs, and internal states. It is customary to distinguish between two models of sequential circuits: the Mealy model and the Moore model. They differ only in the way the output is generated. The two models of a sequential circuit are commonly referred to as a finite state machine, abbreviated FSM.

In the Mealy model, the output is a function of both the present state and the input as shown in Figure 10.1. the outputs may change if the inputs change during the clock cycle. the output of the Mealy machine is the value that is present immediately before the active edge of the clock.

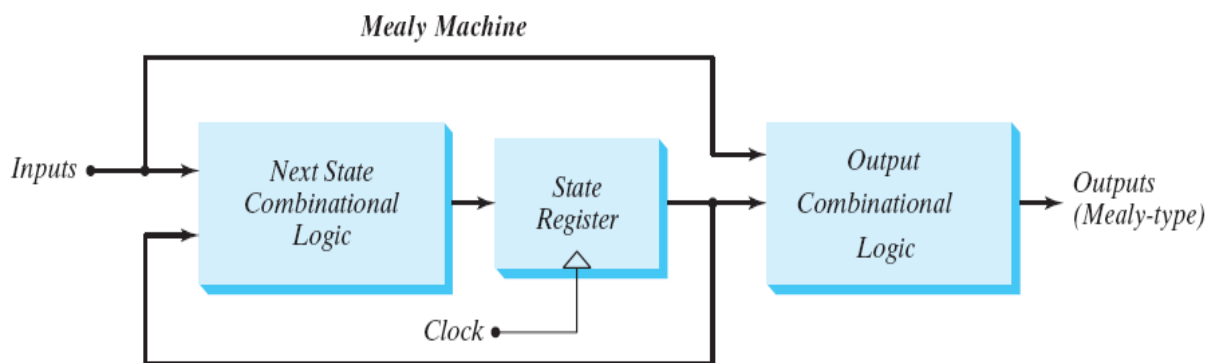


Figure 10.1 Mealy machine model

In the Moore model, the output is a function of only the present state. A circuit may have both types of outputs as shown in Figure 10.2. The outputs of the sequential circuit are synchronized with the clock because they depend only on flip-flop outputs that are synchronized with the clock.

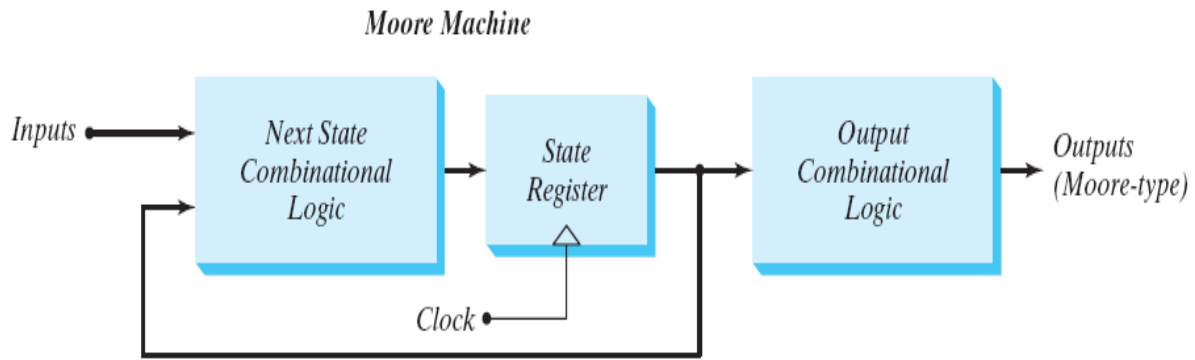


Figure 10.2 Moore machine model

Draw the State Diagram

- Choose the sequence to be detected (e.g., "1011").
- Draw the state diagram for the sequence detector using both Mealy and Moore models.
 - Identify all states and transitions based on the input sequence.
 - For the Mealy machine, specify the output on transitions.
 - For the Moore machine, specify the output on states.

Implement the State Machine in Proteus

- **Create the Circuit:**
 - Use flip-flops (D or JK) to represent the states.
 - Use logic gates (AND, OR, NOT) to implement the state transitions and outputs.
 - For the Mealy machine, connect the outputs to the transitions.
 - For the Moore machine, connect the outputs to the states.
- **Simulation Setup:**
 - Use switches to provide the input bitstream.
 - Use LEDs to display the state of the machine and the detection output.
 - Connect all components according to the state diagram.
- **Simulate the Circuit:**
 - Run the simulation in Proteus.
 - Apply different input sequences and observe the state transitions and outputs.
 - Verify that the output correctly indicates the detection of the specified sequence.

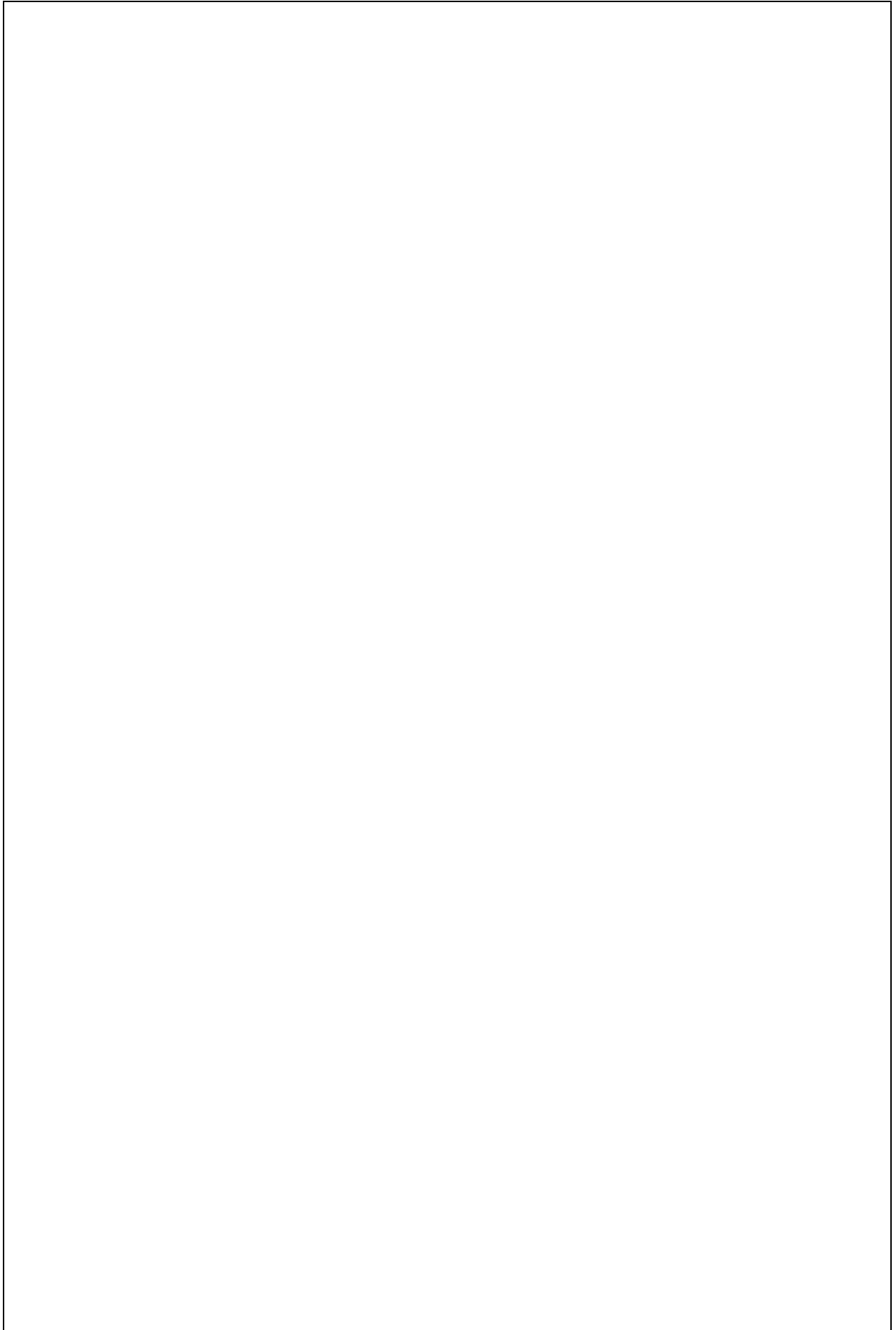
In-Lab: Design and Implementation of a Sequence Detector using Mealy Machine

Sequence: _____

(Note: Sequence will be specified by Lab Instructor)

In-Lab Task 1: State diagram (Mealy), State table, State Equations and Circuit Diagram





Post-Lab Tasks:

1. Using State diagram-based Behavioral model, design and implement on Proteus Mealy based sequence detector for the given sequence.
 - Run simulation for the given task.
 - Record the simulation output.

Critical Analysis/Conclusion

Lab Assessment				
Pre-Lab			/1	/10
In-Lab			/5	
Post-Lab	Data Analysis	/4	/4	
	Data Presentation	/4		
	Writing Style	/4		

Instructor Signature and Comments